

MODULE ARCHITECTURE & DESIGN

PRN & CRBR

Accounts Control

From a site requirement to a signed payment note — through cash, cheque and bank — and into a day book that reconciles itself.

PRN lifecycle

CEO approval

Cash · cheque · bank

Canonical masters

Opening / closing chain

Reconciliation desk



8 failure points today

Duplicate names, site conflicts, balance drift



8 controls that prevent them

6 prevented outright, 2 detected same-day



≈ 3 hours a day returned

Manual work removed from the accounts desk

STACK

Express · Next.js · MongoDB

STATUS

For review

DATE

12 August 2026

What this deck covers



01. The problem

How the flow runs today and the eight points where it loses data.

- 03 The flow today
- 04 Where it breaks
- 05 Design principles



02. In the dashboard

Where the module sits, and the path each role takes through it.

- 06 Target architecture
- 07 Where it lives in raga
- 08 Journey per role
- 09 PRN lifecycle



03. The eight solutions

One control per failure, each preventing rather than reporting.

- 10 Canonical masters
- 12 Allocation & conflicts
- 13 Amendments
- 14 Payments & cheques
- 16 Day book · reconciliation
- 19 Requests · budget



04. Dhwani & the build

What her day becomes, and how the module gets built and rolled out.

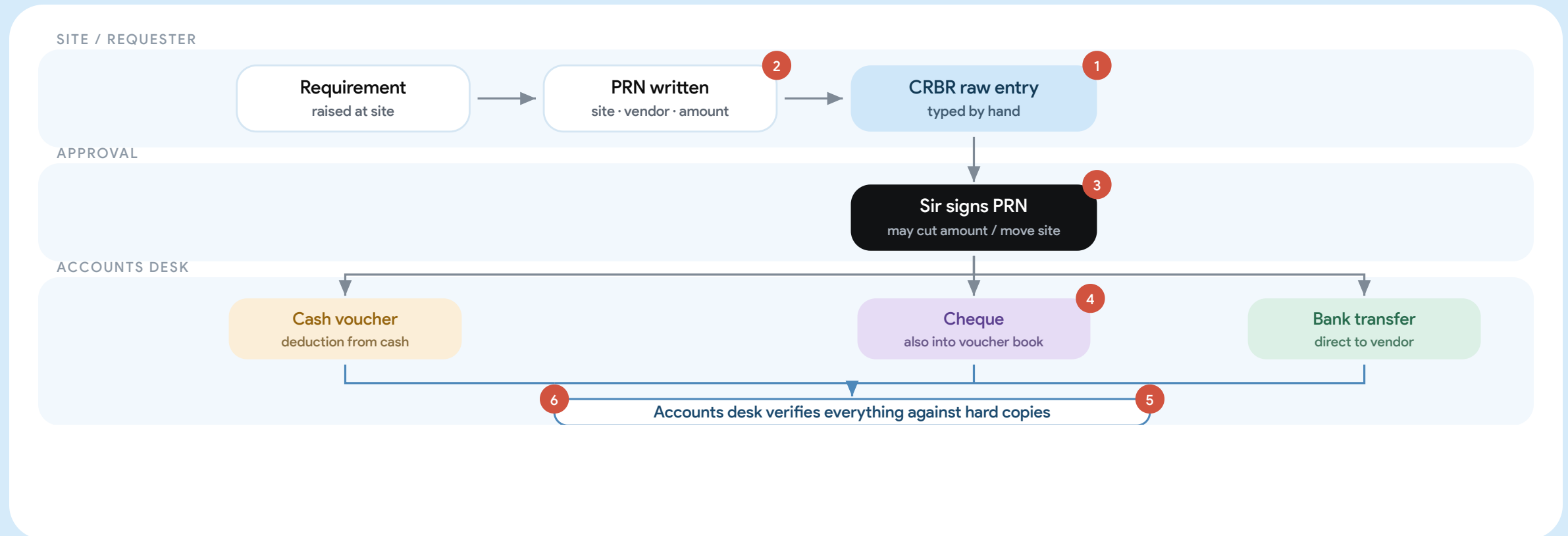
- 21 Her day today
- 22 Her day after
- 23 Manual work removed
- 24 Data model · API
- 26 Migration · rollout



Vocabulary. PRN is the payment request note raised against a site requirement and signed by the CEO. CRBR is the daily cash & bank receipts/payments register it lands in. Voucher book is the physical register every cash voucher and issued cheque is written into. Sir means the approving authority; the accounts desk is the verification role Dhwani performs today.

The flow today

Six hand-offs, three of them on paper and two on spreadsheets nobody controls. Every red marker is a place the data can go wrong with nothing to stop it.



Nothing is a record

The PRN is paper, the voucher book is paper, the CRBR is a sheet anyone can retype. No single object says this payment.



Names are free text

Sites and vendors are typed fresh each time, so **Kalhar**, **KALHAR** and **Kalhaar** become three sites in every rollup.



Verification is memory

One person holds the paper, the sheet, the opening balance and the budget in their head. Nothing is checked twice, or automatically.

Where it breaks

Eight real failures in the current process. Each is answered later by one control — every one of them prevention, not a report that finds the problem afterwards.



S1

01.

Duplicate site names

Typed as `Kalhar`, `KALHAR`, `Kalhaar`, `KALHAAR` — each becomes a new site in totals.

Per-site spend is wrong, and every variant looks plausible.



S1

02.

Duplicate vendor names

Same vendor, different spellings — with and without the trading suffix.

Vendor ledger splits; outstanding balance can't be trusted.



S2

03.

Remark contradicts the site

PRN raised for `Kalhar`, remark says `Paramdham`. Two sites claimed by one payment.

Cost lands on the wrong project; only a reader would notice.



S3

04.

Approval changes invisible

Sir cuts ₹50,000 to ₹30,000, or moves spend to `Paramdham`. Paper is amended; the sheet may not be.

Payment can exceed approval; budget hits the wrong site.



S4

05.

Cheque entered twice

An issued cheque must reach the CRBR and the voucher book — both written by hand.

The two books disagree, and it surfaces days later.



S5

06.

Opening / closing drift

Today's opening is typed from yesterday's closing. One typo, or one back-dated edit, breaks the chain silently.

Every day after is wrong; finding it means re-adding weeks.



S7

07.

Waiting on other people

Figures held by other staff are chased over chat, then filled into a shared sheet.

Day close blocked by a message nobody answered.



S8

08.

Budget known too late

Rates in one sheet, budget in another, spend in the CRBR. Remaining is worked out by hand.

A site overruns before the next PRN can be stopped.



The common root

Seven of these eight are one defect in different clothes: the system accepts text where it should accept a reference. A site typed as a string can be misspelled, contradicted by a remark, or split in a rollup. A site chosen from a registry cannot.

Five rules the module is built on

Every decision that follows comes from one of these. Where a trade-off appears, the rule higher in this list wins.



01.

References, not text

Sites, vendors and materials are stored as an `ObjectId` into a master. The typed text is kept beside it for audit, but nothing is ever grouped or totalled on the raw string.



02.

Derived numbers are never typed

Closing balance, remaining budget and voucher totals are computed on read. If a number can be calculated, there is no field to type it into.



03.

Corrections are reversals

A posted entry is immutable. Mistakes are fixed by a reversal that references the original, so a closed day can never be silently rewritten.



04.

Ambiguity becomes a queue

When the system cannot decide, it does not guess and does not block. It records the entry, flags it, and queues it with the evidence attached.



05.

Paper stays, but stops being the record

Signed PRNs and the voucher book are still checked. The direction reverses: the record is the truth and the check confirms the paper — a tick, not a re-typing.



What this buys the accounts desk

- Name cleanup becomes an occasional approval click, not a daily task
- Opening balance is never typed, so it can never be wrong
- The day cannot close while something is unverified
- Missing figures are requested, tracked, and land in the ledger directly



What it buys management

- Per-site and per-vendor spend correct by construction
- Committed vs actual vs remaining budget, before the next PRN is approved
- A signed record of every amendment — asked, approved, by whom
- Approve from a phone; the change flows to budget and payment



Out of scope for v1

- **Statutory books** — a management register, not a Tally replacement
- **GST & TDS computation** — fields captured, no rates calculated yet
- **Bank statement import** — clearing marked by hand until a feed is agreed
- **PO & goods receipt** — stays with the existing MRN and Stock modules

Target architecture

Four layers, following the shape raga already uses — a guarded Express route, a service that owns the rules, a Mongo collection that stores the result.



SCREENS · NEXT.JS

PRN Desk Approval Inbox Payment Desk CRBR Day Book Voucher Book Masters Budgets **Reconciliation Console** Data Requests Exception Queue Reports & Export



API · EXPRESS

guard → `requireAuth` · `hasMenuAccess(req, "accounts-*")` · `hasPrivilege(...)`

`/api/accounts/prn` `/accounts/payments` `/accounts/crbr` `/accounts/masters` `/accounts/exceptions` `/accounts/budgets`



DOMAIN SERVICES · WHERE EVERY RULE LIVES

EntityResolver

names → ids

AllocationService

splits & conflicts

ApprovalService

signing & amendments

PaymentService

cash · cheque · bank

LedgerService

postings & balances

ReconciliationService

exceptions & day close

BudgetService

committed vs actual

DataRequestService

ask · fill · land



DATA · MONGODB “SHEETS”

accountsPrn accountsPayments accountsLedger accountsDayBook accountsBooks accountsSites accountsVendors accountsMaterials accountsBudgets accountsExceptions

accountsAliasQueue accountsDataRequests



Fits the existing app

Menu ids under an `accounts` parent, exactly how `projects` already groups children. No change to auth or roles.



One writer per number

Only `LedgerService` writes postings, so no two code paths can disagree on a balance.



Sheets become an export

The Sheet stays as a generated view, the way DMR and Stock already mirror — but it is not where data is entered.

Where it lives in raga

One new menu group with six children, sitting alongside Project Control and HR. Nothing existing moves, and users who aren't given the menu never see it.

 raga

Dashboard

Documents

Forms

Project Control

DMR · MRN · Stock · Site Images

HR Dashboard

Employees · Leave · Attendance

 Accounts NEW

PRN

Approvals 3

CRBR

Vouchers

Masters 4

Budgets

Todos

Loop

Reports

Employee Daily Report

Access Control

 The six screens, and what each is for

 PRN

Raise and track payment requests. Site staff see only their own.

 Approvals

Sir's inbox. Card per PRN, built for a phone. Approve, amend, return.

 CRBR

The day book. Opening, postings, computed closing, and day close.

 Vouchers

Generated voucher book plus the cheque tracker, printable for signature.

 Masters

Sites, vendors, materials, rates — and the alias queue that keeps them clean.

 Budgets

Budget, committed, actual and remaining per project and cost head.

 What it plugs into

Project Control	Site and project identity is shared, so accounts spend rolls up to the same projects
MRN · Stock	Material identity is shared — one material master, not three
Loop	Approval and exception notifications ride the existing channel
WhatsApp	The existing reminder runner chases pending approvals and data requests
Forms	Data requests reuse the typed-field builder already in place
Activity Log	Every approval, amendment and day close is written to the existing log

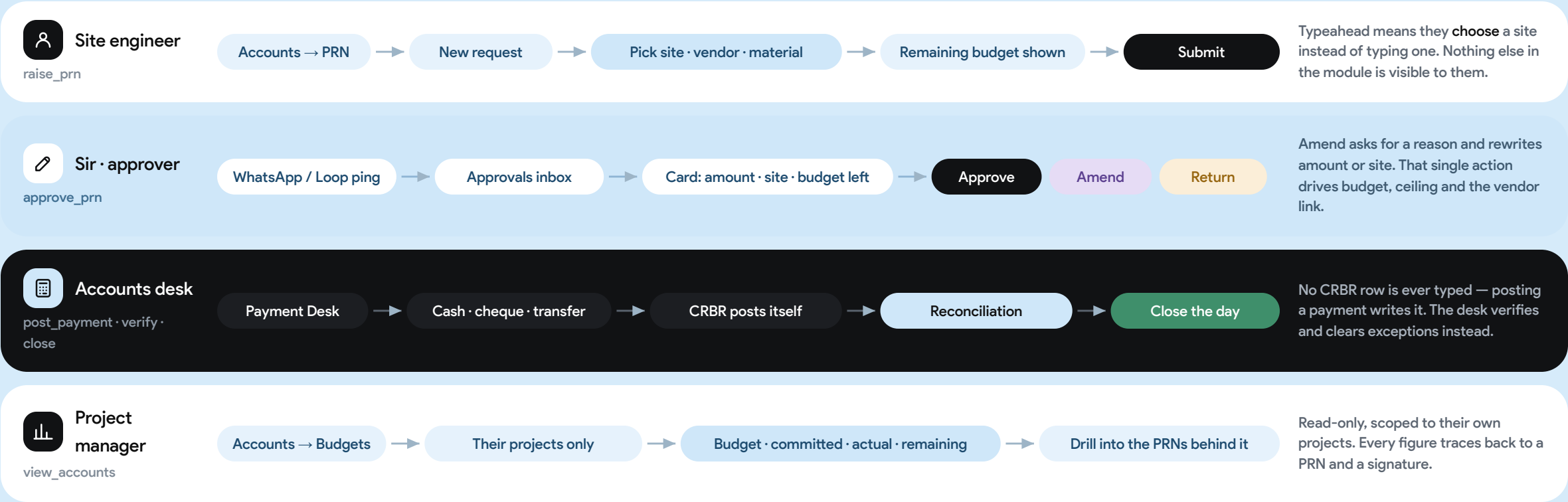
 Badges do the chasing

The sidebar counts are the whole notification design. Nobody has to remember to check a screen — the number appears, and disappears when the work is done.

- Approvals — PRNs waiting on Sir, red past a day
- Masters — names in the alias queue awaiting a decision
- CRBR — days still open, and exceptions blocking close

The journey, role by role

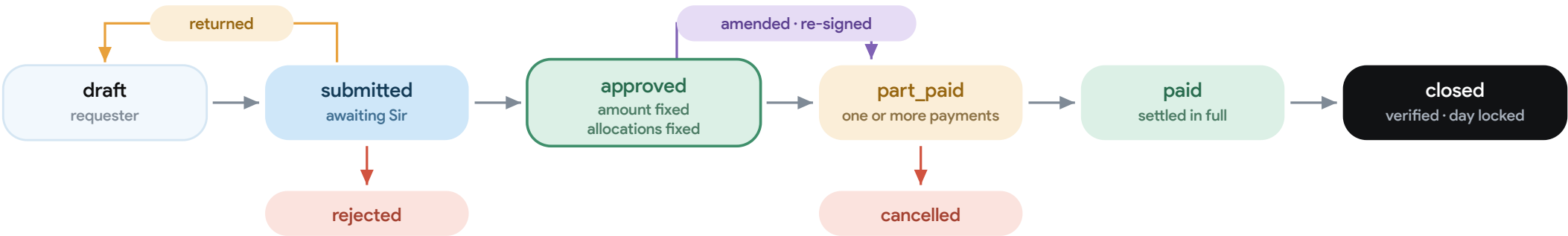
The same PRN seen from four seats. Each row is the actual click path through the dashboard, from the moment the work arrives to the moment it leaves.



⚡ The handover points are where the value is. Today each arrow above is a phone call, a WhatsApp photo of a paper PRN, or a row retyped into a sheet. In the module every arrow is a state change on one object — which is why the same PRN can be traced end to end, and why nobody has to ask where it got to.

PRN lifecycle

Every other record hangs off the PRN. A payment cannot exist without an approved PRN behind it, and a CRBR line cannot exist without a payment — that chain is what makes the register verifiable.



TRANSITION GUARDS — ENFORCED IN THE SERVICE LAYER, NOT THE UI
submit needs every name resolved to an id · a payment needs status = approved and must stay inside approvedAmount · close needs verification ticked and zero open exceptions



Who moves it

TRANSITION	ROLE	PRIVILEGE
draft → submitted	Site / requester	raise_prn
submitted → approved	CEO or delegate	approve_prn
approved → amended	CEO only	approve_prn
approved → paid	Accounts	post_payment
paid → closed	Accounts desk	close_accounting_day

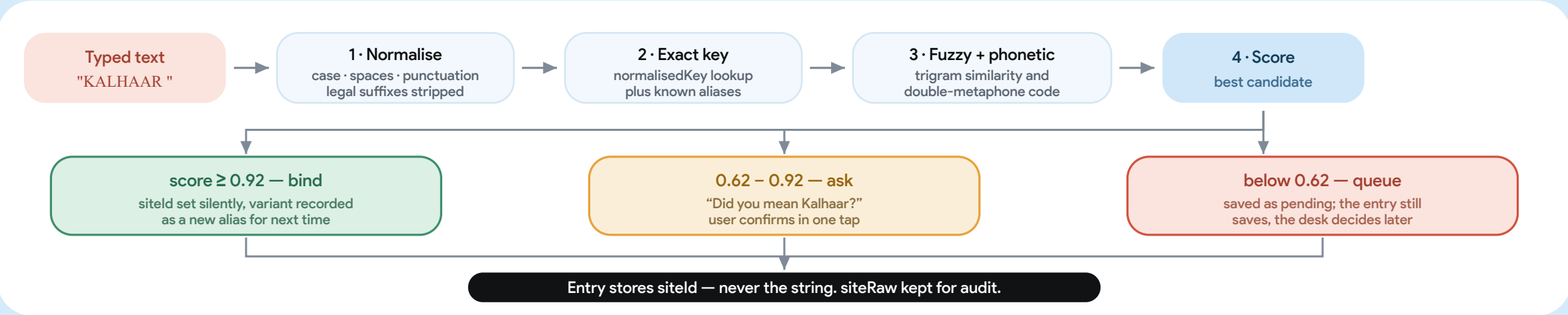


What each PRN carries

Number	PRN/26-27/0148 — issued, never typed
Lines	Material, qty, rate, amount — each resolved
Allocations	Site shares that must total the amount
Approval	Approved amount, allocations, signature, time
Amendments	Ordered diffs — actor, reason, timestamp
Payments	Back-references from every settlement

Kalhar = KALHAR = Kalhaar, permanently

Not a cleanup script that runs nightly — the duplicate is made impossible to create. The form binds to a master record, and text that doesn't resolve goes to a queue instead of quietly becoming a new site.



Worked through the real variants

TYPED	NORMALISED	PHONETIC	SCORE	OUTCOME
Kalhar	kalhar	KLHR	1.00	Exact key — bound, no interaction
KALHAR	kalhar	KLHR	1.00	Case folds away — same key, same site
Kalhaar	kalhaar	KLHR	0.94	Phonetic hit — auto-bound, alias saved
KALHAAR	kalhaar	KLHR	0.94	Space stripped — hits the new alias
Kalher	kalher	KLHR	0.88	Below bind — user is asked to confirm
Kalpataru	kalpataru	KLPTR	0.34	Genuinely different — offered as new



Why phonetic, not just edit-distance

Edit-distance treats `Kalhar` / `Kalhaar` as one character apart — but it treats genuinely different names as equally close. A double-metaphone code collapses the vowel-doubling and transliteration variants that dominate Gujarati and Hindi site names, which is exactly the duplicate class in the live data.



It gets quieter over time

Once `Kalhaar` resolves to `Kalhar` even once, the alias is stored — the second person to type it never sees a prompt. Thresholds are configuration, tunable per entity from the Masters screen.

One record per real-world thing

Four masters, one shape: canonical name, normalised key, aliases, merge history. Merging two that turn out to be the same is a first-class operation, not database surgery.

accountsSites

master

```
name string "Kalhar" — shown everywhere
code string "KAL-01"
normalisedKey string unique index
phoneticKey string "KLHR", indexed
aliases[] string[] every accepted variant
projectId string → projectDashboard
mergedInto ObjectId? set when superseded
```

accountsVendors

master

```
name string
normalisedKey string unique index
aliases[] string[]
gstIn, pan string strongest dedupe signal
bank{...} object for transfer payments
siteIds[] ObjectId[] many-to-many
```



The many-to-many that matters

A vendor is not owned by a site. `siteIds[]` is a list precisely because the same vendor supplies the same material to several projects — which is what makes it legitimate for Sir to move a PRN from Kalhar to Paramdham without changing the vendor. Without this, the amendment on the next page cannot be represented at all.



Merging duplicates that already exist

01.

Pick survivor

A preview shows how many PRNs, payments and ledger lines each side carries.

02.

Repoint, don't rewrite

The loser gets `mergedInto`; old documents keep the id they were written with.

03.

Reversible

Nothing was destroyed, so a wrong merge is undone by clearing one field.



The alias queue

3 waiting

Everything the matcher couldn't decide, with the candidates it considered. Three buttons: **bind**, **create new**, **reject**. This is the only manual name work left — clicks per week, not hours per day.

TYPED	SEEN IN	CANDIDATES	ACTION
SILVER WHITE FTY	PRN/O148	Silver White Factory 0.71	Bind
Devsharnam	CRBR 11 Aug	Devsharanam 0.91	Bind
Shreeji Hardware	PRN/O151	Shreeji Traders 0.55	Create new

The site in the remark vs the site on the PRN

Today a PRN says Kalhar and its remark says Paramdham, and only a careful reader notices. The cause is that the remark is the only place a second site can be expressed.

Part one — allocations make the split expressible

A PRN carries `allocations[]` instead of a single site. Most have exactly one entry and the form looks no different. When work genuinely spans two projects, the requester says so precisely — and the amounts must total the PRN or the API rejects it.

```
siteId ObjectId req
projectId string
amount number req
costHead string
```

₹50,000 STEEL, TWO SITES

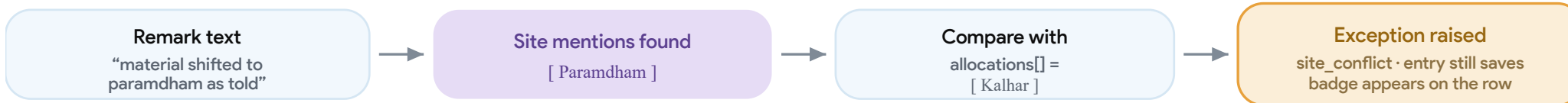
Kalhar	₹30,000
Paramdham	₹20,000
PRN total	₹50,000

Deliberately noisy in one direction

A remark naming a site already in the allocations produces nothing. A remark naming a site that is **not** always produces a flag — even when innocent, like “collected from Kalhar office” on a Paramdham PRN.

A false flag costs one click. A missed misallocation costs a wrong project cost that nobody ever finds.

Part two — the remark is scanned, and disagreement becomes an exception



THE DESK THEN CHOOSES ONE OF THREE — ONE CLICK, AND THE REASON IS RECORDED

Reallocate to Paramdham

Split across both

Remark is informational — dismiss

When Sir cuts the amount or moves the site

A PRN for ₹50,000 at Kalhar comes back approved for ₹30,000 at Paramdham, because that project needs to move. This is normal and legitimate — the problem is only that it happens on paper.

REQUESTED — FROZEN AT SUBMIT

amount	50,000
vendor	Shreeji Traders
material	TMT bar 12mm
site	Kalhar



signs

APPROVED — THE BINDING FIGURES

amount	30,000
vendor	Shreeji Traders
material	TMT bar 12mm
site	Paramdham

AMENDMENT RECORD

amount	50,000 → 30,000
site	Kalhar → Paramdham
reason	“push Paramdham work first”
by	CEO · 12 Aug 2026, 09:42



What that one action drives automatically

Kalhar's committed budget releases ₹50,000 · Paramdham's commits ₹30,000 · the payment ceiling drops to ₹30,000 · the vendor-site link is created if this is their first Paramdham order. Nobody re-keys a budget sheet.



Rules the service enforces

- ✓ Reducing an amount is free. **Increasing** beyond the request is allowed but always recorded with a mandatory reason.
- ✓ Reallocating to another site is permitted only if the vendor serves that site for the material, or the approver confirms creating the link.
- ✓ Amended allocations must still total the approved amount.
- ✗ No payment may exceed `approvedAmount` ; a part-paid PRN cannot be amended below what has already gone out without a reversal first.
- ✗ Nobody but the approver can write approval fields, whatever their menu access.



Approving from the phone

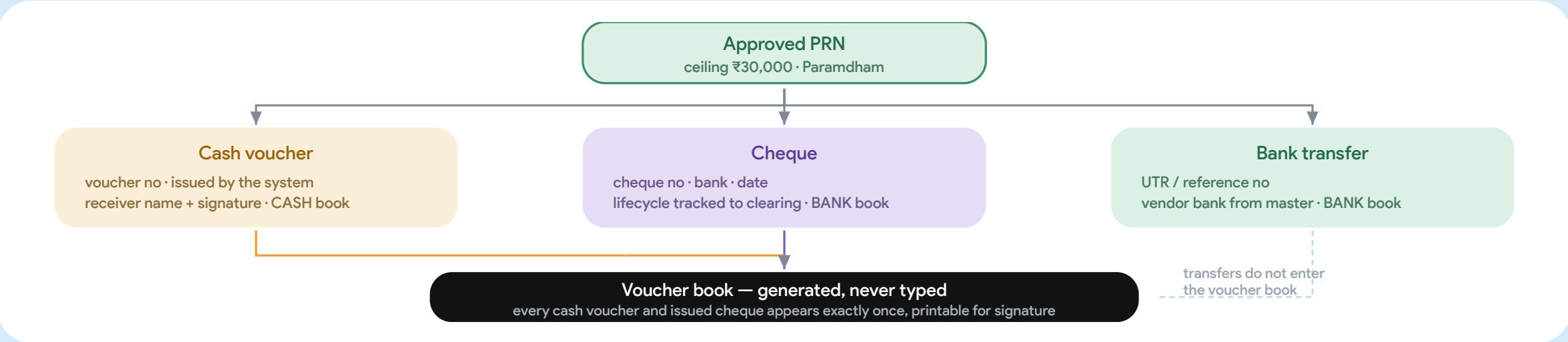
A list of cards — requester, site, vendor, amount, and the remaining budget for that site. Three actions:

- Approve** signs as requested
- Amend** opens amount and allocation, requires a reason
- Return** sends it back with a note, no rejection stigma

The signature is the approver's authenticated action plus timestamp. The paper PRN can still be signed — the record no longer depends on it.

Cash, cheque, bank transfer

Three instruments, one payment object. The instrument decides which book the posting lands in and what evidence is required — the shape of the record never changes.



Evidence each instrument demands before it can post

INSTRUMENT	BOOK	REQUIRED	VOUCHER BOOK
Cash voucher	Cash	Voucher no, receiver name, amount inside the ceiling, and available cash on that date	Yes
Cheque	Bank	Cheque no, bank account, date, payee resolved to the vendor master — number unique per account	Yes
Bank transfer	Bank	UTR or reference, vendor bank details taken from the master rather than typed	No

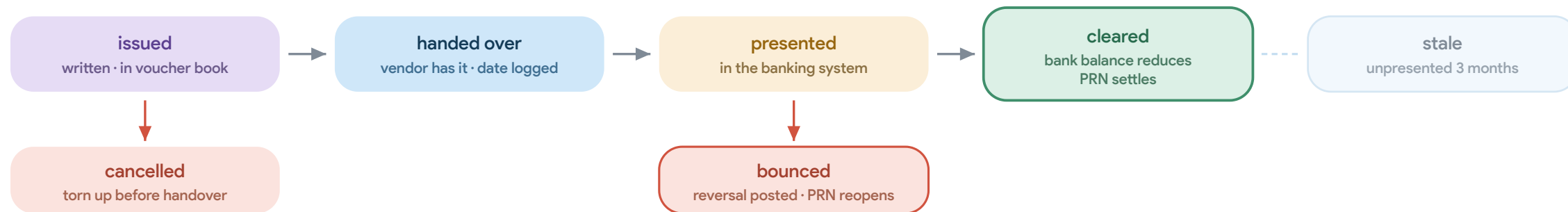
Two invariants that make this safe

- One.** A cheque number is unique within a bank account — the index enforces it, so the same cheque cannot be recorded twice under any circumstances.
- Two.** The sum of payments against a PRN can never exceed its approved amount, checked inside the same transaction that writes the payment — so two people posting at once cannot both slip through.

A cheque is not cash until it clears

Cash and transfers settle the moment they happen. A cheque is written on one day, handed over on another, presented later, and may bounce — treating it as settled on the day it was written is the most common reason a bank book stops agreeing with the bank.

EACH TRANSITION CARRIES A DATE — WHICH IS WHAT MAKES BANK RECONCILIATION POSSIBLE



Two balances, both true

Bank book balance	₹4,20,000
Less: cheques issued, not cleared	- ₹85,000
Available balance	₹3,35,000

The desk sees what the bank will say and what is actually spendable, side by side, with no manual working.



A bounce is a reversal, not an edit

The original payment is never deleted or amended. A reversal is posted on the date the bank returned it, referencing the original — and the effects follow:

- Bank book shows the debit and the reversal on their real dates
- The PRN drops back from `paid` to `part_paid`
- Committed budget is restored to the allocated sites
- The vendor's outstanding balance reappears; bank charges post as their own line
- Days already closed are never touched



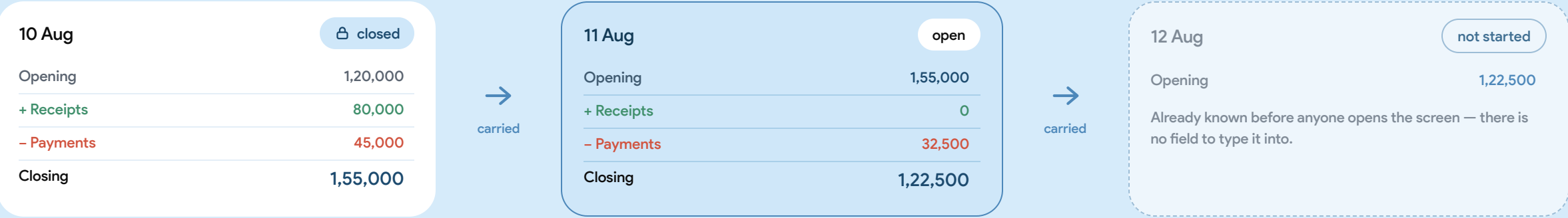
The field that saves the phone calls

Handed over — the moment the vendor physically receives the cheque — is captured with a date, the person who handed it over, and optionally an acknowledgement.

That single field answers the question that currently costs the most phone calls: **has the vendor got the cheque, or is it still with us?**

Opening, closing, and the chain between them

“Yesterday's closing is today's opening” currently describes a number somebody re-types every morning. Here it describes a rule the database enforces, so the chain cannot break.



$\text{closing}(d) = \text{opening}(d) + \text{receipts}(d) - \text{payments}(d)$
 $\text{opening}(d) = \text{closing}(d-1)$

– computed on read, never an editable number

– resolved by reference, so it cannot be typed differently

What a day book row actually is

Every row is produced by something else — nothing is typed straight into the register, which is what makes each line traceable to a PRN and a piece of paper.

ROW	CREATED BY	TRACES TO
Payment	Cash, cheque or transfer against an approved PRN	PRN + voucher no
Receipt	Client payment, cash withdrawal, refund	Receipt no
Contra	Bank to cash — one out, one in	Both books
Reversal	A bounce or correction, dated when it happened	Original posting

The failure this removes

Today a back-dated edit to a closed day changes that day's closing but not the next day's opening — because the opening was typed. The two silently disagree, and every balance after inherits the error.

Here the opening is not a stored number at all — so there is nothing that can disagree.

Closing a day locks its postings. Corrections become a reversal dated today. Reopening is possible, but it is privileged and logged — not a click into a cell.

The verification desk, made visible

This is the screen Dhwani lives in. Not a report — a worklist. Everything the system could not decide arrives here with the evidence attached, and the day cannot close until the list is empty or every remaining item has been explicitly accepted.

Reconciliation — 11 August 2026

⚠️ 4 open exceptions

✓ 7 cleared today

EXCEPTION QUEUE

Site conflict

PRN/0148 allocated to Kalhar · remark names Paramdham

Resolve

Unresolved vendor

“Shreeji Hardware” — closest match Shreeji Traders at 0.55

Decide

Awaiting data

Site petty cash 11 Aug — requested from R. Patel, 1 day overdue

Remind

DAY CLOSE CHECKLIST

✓

Cash counted matches book

✓

All PRNs seen in hard copy

✓

Voucher book signed

◯

Exceptions cleared

4 remaining

◯

Opening tallies with 10 Aug automatic — nothing to do

The five exception types

TYPE	RAISED WHEN	RESOLUTION
unresolved_name	A site, vendor or material could not be matched above the confidence threshold	Bind, create new, or reject
site_conflict	Free text names a site that is not among the PRN's allocations	Reallocate, split, or dismiss
rate_variance	A line rate deviates from the material master beyond the configured tolerance	Accept with reason, or amend
awaiting_data	A data request needed for this date is still unanswered	Remind, or close without it
doc_missing	A posting has no hard-copy verification recorded against it	Tick after sighting the paper

raga · accounts — PRN & CRBR architecture

17

Checking the folder against the record

The physical check does not go away — it catches a payment made without a signed PRN, which no software can detect on its own. What changes is that Dhvani confirms a record instead of transcribing paper into a sheet.

posting.verification

embedded

```
prnSighted bool signed PRN in the folder
voucherSighted bool voucher / cheque counterfoil
signatureSeen bool approver's signature present
docNumbers[] string[] as written on the paper
amountMatches bool paper figure = record figure
verifiedBy ObjectId
verifiedAt Date
note string only when something differs
```



Why a difference is never auto-adjusted

A cash count that disagrees with the book by ₹500 is a real event with a real cause — an unrecorded payment, a miscount, or money genuinely missing. Writing a balancing figure to make the two agree destroys the only signal that something happened.

The module records the count, records the difference, and requires someone to say what it was before the day can close. If `amountMatches` is false, the posting is **not** corrected in place — the desk records what the paper says, and the gap becomes an exception for someone to explain.



How the desk works through it

1

Open the day, sorted by voucher number

The screen lists postings in the order the folder is filed, so the desk works down the pile without searching for anything.

2

Tick each line against its paper

One row, four checkboxes, keyboard-driven. The document number is pre-filled from the record and confirmed rather than typed.

3

Anything unusual becomes an exception

Missing paper, wrong amount, missing signature — each opens an exception with the posting attached, rather than a note nobody sees again.

4

Cash count closes the loop

The desk enters the physical cash counted. The system shows the difference against the computed closing — and never adjusts the book to match.



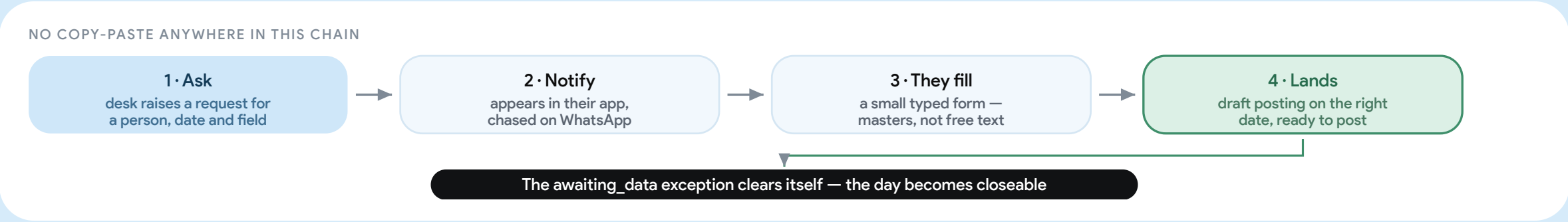
Dual control, for the days it matters

Because verification is a field with a user id on it, a second reviewer can be required without changing any process — payments above ₹1,00,000 need a second verification, or month-end close needs the accounts head. The second person sees exactly what the first ticked, stored separately.

Today this is impossible: there is one folder, one sheet, and one person's memory of having checked it.

Replacing “please fill the sheet”

Some figures Dhwani needs are held by other people — site petty cash, a bill that hasn't arrived, a labour count. Today that is a message, a shared sheet, and a wait.



Why this beats the shared sheet

SHARED SHEET TODAY	DATA REQUEST IN THE MODULE
Asked over chat, no record	A record with requester, requestee, date, field and due date
Nobody knows what is outstanding	A list of open requests, with overdue ones surfaced
Free text, so names re-duplicate	Same resolver as everywhere else — the answer arrives already bound to masters
Desk copies it into the CRBR	The answer becomes a draft posting on the correct date
Silent if it never arrives	Blocks day close, and chases automatically

Reuse, not new plumbing

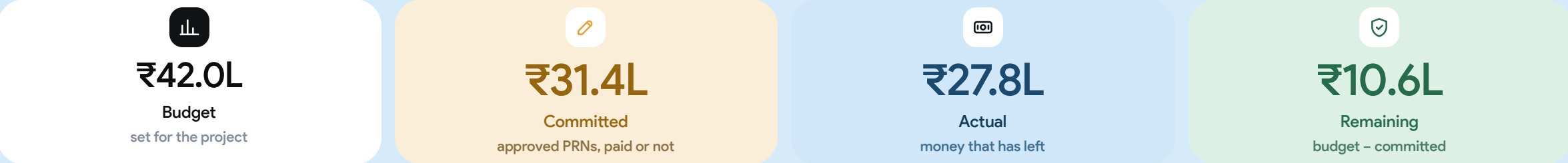
This is deliberately thin, because raga already has the pieces:

- Forms already defines typed fields and stores submissions
- WhatsApp reminders already chase people on a schedule
- Loop already delivers in-app notifications

A data request is a small record tying those three together and routing the answer into the ledger. Little of it is new code.

Committed, actual, remaining

Rates and budgets stop being separate sheets someone reconciles afterwards. Both become masters the PRN reads at approval — the only moment knowing the remaining budget can change the outcome.





Remaining is measured against committed, not actual

A PRN approved yesterday and unpaid today has already spent the budget in every sense that matters — the commitment exists and the vendor will be paid. Subtracting only what has physically left the account overstates what is available, and is exactly how a site overruns while the sheet still looks healthy.

 Where the check happens

MOMENT	WHAT THE SYSTEM DOES
PRN drafted	Shows remaining budget for each allocated site, live, beside the amount field
Submitted	Warns if this PRN would take the site past its budget — does not block; the approver decides
Approval	Remaining budget is on the card. Approving past it requires a reason, stored on the amendment
Amendment	Commitment moves between sites automatically when allocations change
Payment	Actual increases; committed is unaffected — it was already counted at approval

accountsMaterials.rates[]

effective-dated

```
vendorId ObjectId? null = general rate
siteId ObjectId? null = all sites
rate, unit number kg · bag · nos · sqft
effectiveFrom string "YYYY-MM-DD"
effectiveTo string? null = current
```



Variance is a flag, never a block

Rates are dated rather than overwritten, so a March PRN is checked against March's rate and the most specific match wins. A line 10% above the master raises a `rate_variance` exception showing both figures — prices move, so the desk accepts it with a reason. Accepting also offers to update the master, which is how the rate sheet stays current without anyone maintaining it separately.

Dhwani's day today

Seven activities, all judgement applied to unstructured text — unrepeatable, unauditable, and impossible to hand to a second person.



Collect & match

CRBR rows ↔ hard copies

Pull the day's raw CRBR rows and match each to the physical PRN and voucher in the folder, by hand, one at a time.

≈ 45 min

depends on the folder being complete



Clean the names

the single biggest task

Read every site and vendor name, decide which spellings mean the same thing, and correct them — Kalhar / KALHAR / Kalhaar / KALHAAR all over again, every day.

≈ 40 min

lives only in her head



Chase site conflicts

PRN says one site, remark another

Spot that a PRN raised for Kalhar has Paramdham in the remark, then ask someone which one it really was.

≈ 20 min

only caught if she reads it



Chase other people

figures she doesn't hold

Message whoever holds the missing figures, wait for them to fill the shared sheet, then copy it across.

≈ 30 min

plus dead waiting time



Balance the day

opening, closing, re-add

Carry yesterday's closing forward as today's opening, then re-add the day until the two sides agree. If they don't, work backwards through the week.

≈ 35 min

depends on yesterday being right



Voucher book

written a second time

Write every cash voucher and issued cheque into the physical register, then cross-check it against the CRBR to find what was missed.

≈ 25 min

the same data, twice



Rates & budget

three sheets, kept in step

Cross-check spend against the materials rate sheet and the project budget sheet, and work out what is left, by hand.

≈ 30 min

after the money has gone



≈ 3h 45m
per working day

Plus rework whenever an error surfaces days later. These are indicative estimates from the process as described — worth confirming with Dhwani before the numbers are quoted anywhere, but the **shape** is what matters: almost all of it is transcription and cross-checking, not accounting judgement.

Dhwani's day with the module

Four of the seven activities disappear entirely. What remains is the part that genuinely needs her judgement — confirming the paper, and deciding the questions the system was right not to guess at.

✓ Gone completely

Cleaning names

–40 min

Entry binds to a master. The variant never gets created, so there is nothing to clean.

Balancing the day

–35 min

Opening is resolved from yesterday's closing. Closing is computed. Neither can be typed or drift.

Writing the voucher book

–25 min

Generated from the payments themselves. She prints it and collects signatures.

Rate & budget cross-check

–30 min

Committed, actual and remaining are live. Variance comes to her as a flagged line.

↻ Reduced to a queue

Collect & match

45 → 20 min

Rows are already there, in folder order, each linked to its PRN. She ticks rather than transcribes.

Site conflicts

20 → 5 min

Found automatically and queued with both sites shown. One click to reallocate, split or dismiss.

Chasing other people

30 → 5 min

She raises a request once. The system notifies, chases, and lands the answer as a draft posting.

Unknown names

new · 5 min

The alias queue — a handful of decisions a week, each with the candidates and scores shown.

✎ What stays with her — by design

Confirming the paper exists

Only a person can confirm a signed PRN is physically in the folder. It becomes a tick against a record, not a re-typing.

Deciding what a new name means

The system ranks candidates but cannot know a genuinely new vendor isn't a misspelling. That judgement is hers.

Explaining a cash difference

When the count disagrees with the book, the cause is in the room. The module records it and refuses to paper over it.

Signing off the day

Closing the day stays a deliberate act with her name on it. The checklist shows what is outstanding; she decides when it is done.

≈ 35–40 min

per working day

Roughly three hours returned each day — and, more importantly, the work that remains can be handed to a second person, because it is a checklist against records rather than knowledge held in one head.



The point is not that she works less. It is that the day **cannot silently go wrong** while she is doing something else.

Manual work the module removes

Not only at the accounts desk. Every row is work that exists today purely because the record is paper and the register is a spreadsheet.



At the accounts desk

MANUAL WORK TODAY	WHAT REPLACES IT	OUTCOME
Correcting site and vendor spellings, every day	Entry binds to a master; only genuinely new names reach the queue	Gone
Carrying the opening forward and re-adding the day	Opening resolves from yesterday's closing; closing computed on read	Gone
Writing the voucher book, then reconciling it	Generated from the payments and printed for signature	Gone
Cross-checking three sheets for rates and budget	Committed, actual and remaining live; variance arrives as a flag	Gone
Reading remarks to catch a mismatched site	Scanned on save; mismatches queue with both sites shown	Less
Transcribing rows against the hard copies	Rows exist in folder order; verification is a tick plus a doc number	Less



Everyone else

WHO	MANUAL WORK TODAY → WHAT REPLACES IT	OUTCOME
Site staff	Filling a paper PRN and sending a photo onward → a typed request with pickers for site, vendor and material	Gone
Site staff	Being asked twice which site it was really for → allocation captured when raised, and it can span two sites	Gone
Other staff	Chased on WhatsApp to fill a shared sheet → a tracked request with a due date, answered in a small form	Less
Sir	Signing paper, then relying on someone to reflect it → approve or amend on the phone; budget and ceiling follow	Gone
Managers	Asking the desk what is left in a budget → a read-only screen scoped to their projects, always current	Gone
Everyone	Rework when an error surfaces days later → caught at entry or day close, while the context is fresh	Less

8

activities removed outright

4

reduced to a checklist or queue

≈ 3h

returned to the accounts desk daily



The relief is real but secondary. The primary gain is that the register becomes **verifiable** — every figure traces to a PRN, a signature and a piece of paper, and no single person has to hold it together.

Collections and their shape

Twelve new collections in the existing `sheets` database, following the conventions the app already uses — and deliberately correcting two traps the current schema carries.



Conventions kept

Business dates are "YYYY-MM-DD" strings so range filters stay lexical · timestamps (`createdAt` , `postedAt` , `verifiedAt`) are real `Date` objects · state is soft, documents are not deleted



Traps deliberately avoided

Every `userId` here is an `ObjectId` — never the hex string `employeeDailyReports` uses, so joins need no conversion · names are never the grouping key, `siteId` is

accountsPrn

the request

```
prnNo string "PRN/26-27/0148" unique
status string draft ... closed
requestedBy ObjectId → users
requestDate string "YYYY-MM-DD"
vendorId ObjectId
vendorRaw string as typed, audit only
lines[] object[] materialId, qty, rate
amount number = sum(lines.amount)
allocations[] object[] siteId, amount, costHead
remark string
approval object? approvedAmount + allocs
amendments[] object[] field, from, to, reason
paidAmount number kept by PaymentService
```

accountsPayments

the settlement

```
prnId ObjectId required
instrument string cash|cheque|transfer
bookId ObjectId which cash/bank account
paymentDate string "YYYY-MM-DD"
amount number
voucherNo string? cash + cheque only
cheque object? no, bank, date, state,
                                     handedOverAt, clearedAt
utr string? transfers
verification object see page 18
reversalOf ObjectId?
postedBy ObjectId
postedAt Date
```

accountsDayBook

one per book per day

```
bookId ObjectId
date string unique with bookId
status string open | closed
receiptsTotal number derived on close
paymentsTotal number derived on close
closingBalance number snapshot, written once
cashCounted number? physically counted
countDiff number? counted - computed
countNote string? required if diff ≠ 0
checklist object the five close gates
closedBy ObjectId?
reopenedLog[] object[] by, at, reason
```



Indexes — created in `connectAuthDb`, beside the existing ones

```
accountsPrn { prnNo:1 } unique · { status:1, requestDate:-1 } · { "allocations.siteId":1, requestDate:-1 }
accountsPayments { prnId:1 } · { bookId:1, paymentDate:1 } · { "cheque.bankAccountId":1, "cheque.no":1 } unique sparse ← a
duplicate cheque becomes impossible
accountsDayBook { bookId:1, date:1 } unique ← the chain's backbone · accountsSites / accountsVendors { normalisedKey:1 }
unique, { phoneticKey:1 }
```



Why `openingBalance` is not a field

Its absence is the entire point of Solution 5. Opening resolves as the `closingBalance` of the last closed day for that book — it cannot be typed, edited, or drift. Closing is stored, but only at the moment of close.

Endpoints, menus, privileges

Routes follow the pattern already in `backend/server.js` — authenticate, check menu access, check privilege, delegate to the service. No business rule lives in a route handler.

 API surface

VERB	PATH	PURPOSE	PRIVILEGE
GET	/accounts/prn	List with filters — status, site, vendor, dates	view_accounts
POST	/accounts/prn	Create a draft; resolves names, returns confidence	raise_prn
POST	/accounts/prn/:id/submit	Validates allocations total and all ids resolved	raise_prn
POST	/accounts/prn/:id/approve	Writes approval; body may carry the amendment	approve_prn
POST	/accounts/payments	Post cash, cheque or transfer; ceiling enforced in a transaction	post_payment
PATCH	/accounts/payments/:id/cheque	Advance cheque state — handed, presented, cleared, bounced	post_payment
POST	/accounts/payments/:id/reverse	Post a reversal; never edits the original	post_payment
GET	/accounts/crbr/:bookId/:date	Day view — opening resolved, postings, computed closing	view_accounts
POST	/accounts/crbr/:bookId/:date/close	Runs the checklist; refuses while exceptions are open	close_accounting_day
POST	/accounts/masters/:entity/resolve	Behind every typeahead — returns ranked candidates	view_accounts
POST	/accounts/masters/:entity/merge	Merge duplicates by repointing, never rewriting	manage_accounts_masters
GET	/accounts/budgets/:projectId	Budget, committed, actual, remaining per cost head	view_accounts

MENU_ITEMS additions

```
{ id: "accounts", label: "Accounts", group: "accounts" }
{ id: "accounts-prn", parent: "accounts" }
{ id: "accounts-approvals", parent: "accounts" }
{ id: "accounts-crbr", parent: "accounts" }
{ id: "accounts-vouchers", parent: "accounts" }
{ id: "accounts-masters", parent: "accounts" }
{ id: "accounts-budgets", parent: "accounts" }
```

 Who gets what

Site engineer	PRN only · raise_prn — sees only their own
Sir	Approvals + Budgets · approve_prn
Dhwani	All six · everything except approve_prn
Accounts head	All, including reopening a closed day
Project manager	Budgets · view_accounts , scoped to their projects

Menu access controls whether a screen is reachable; privilege controls what can be done there.

Bringing the existing CRBR across

The historic sheet is exactly the raw, duplicated data the module is designed to prevent — so the import is also the first and largest run of the resolver. Done once, reviewed, and it does not need to be perfect to be useful.



01.

Import raw, unchanged

Every historic row lands with `narrationRaw` and the original site and vendor strings intact. Nothing is corrected yet — a faithful copy first, so the import can always be re-run.



02.

Build masters from distinct values

Cluster every distinct string by normalised and phonetic key, proposing one master per cluster with the most frequent spelling as canonical.



03.

Review before committing

Clusters are shown with their member spellings and row counts. Confirm, split or rename. This is the largest piece of human work in the migration — and it happens once.



04.

Bind the history

Every imported row is re-scanned and bound to ids. Rows that still won't resolve keep their raw text and are tagged — history is for reporting, not re-auditing.



05.

Set the opening balance once

One `opening` posting per book on go-live, signed off against the physical count and the bank statement. The only opening balance anyone will ever type.



Expect this from the import

- Roughly 5–15% of rows will need a human decision on names
- Some historic rows have no PRN reference at all and never will — they import as ledger rows without a `prnId`, allowed for imported history only
- Opening balances before go-live are not reconstructed; history is for reporting, not for chaining
- Expect the clustering to over-merge in a few places — that is what the review step is for

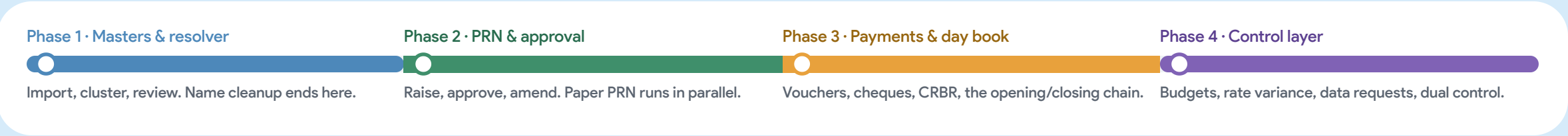


What makes it safe

- The raw import is idempotent and re-runnable — clustering can be redone without touching the source
- Masters are reviewed by a person before anything binds to them
- A wrong cluster is fixed later by merge, which is reversible
- The live system never depends on historic rows being perfect — only on go-live forward being correct

Four phases, each useful on its own

Ordered so the earliest phase already removes the most painful daily work. Nothing here needs the whole module finished before the desk feels a difference.



PHASE	SHIPS	THE DAY IT GOES LIVE	RUNS ALONGSIDE
1	Masters & resolver sites, vendors, materials, alias queue, import	Dhwani stops correcting spellings by hand. Per-site and per-vendor totals become correct for the first time, including historic data. Nothing else about the process changes yet.	Existing sheet, unchanged
2	PRN & approval lifecycle, amendments, allocations, conflict detection	Requests are raised in the app and approved from a phone. Amendments become visible and auditable, and the site-conflict flag starts catching remark mismatches.	Paper PRN still signed
3	Payments & day book instruments, cheque lifecycle, CRBR, voucher book, verification	The register moves into the module. Opening balance stops being typed and the voucher book becomes a printout. This phase needs a clean go-live date and a signed-off opening balance.	Sheet becomes an export
4	Control layer budgets, rate variance, data requests, dual control	Remaining budget is visible at approval rather than discovered later. Requests for missing figures are tracked and chased. Optional second verification on large payments.	—



The sequencing decision worth keeping

Masters ship first even though they are the least visible piece. Every later phase depends on names already resolving — building PRN entry before the resolver exists would mean redoing the forms and re-importing the data. It also means the very first release, on its own, deletes the daily task that costs the accounts desk the most time.

Every problem, and the control that stops it

The eight failures from page 4, each mapped to the mechanism that prevents it and the phase in which that mechanism ships.

#	PROBLEM	THE CONTROL	TYPE	PHASE
01	Duplicate site names	Entry binds to <code>siteId</code> ; unmatched text goes to the alias queue instead of creating a site	Prevented	1
02	Duplicate vendor names	Same resolver, plus GSTIN and PAN as exact-match signals where available	Prevented	1
03	Remark contradicts the site	<code>allocations[]</code> makes a genuine split expressible; the remark scanner flags anything else	Detected	2
04	Approval changes invisible	Approval is a separate block from the request, with an amendment diff carrying actor and reason	Prevented	2
05	Cheque entered twice	Voucher book generated from payments, not typed; unique index on cheque number per bank account	Prevented	3
06	Opening / closing drift	No <code>openingBalance</code> field exists — it resolves to the prior day's closing; closed days lock	Prevented	3
07	Waiting on other people's data	Data requests with due dates that block day close and chase automatically	Detected	4
08	Budget known too late	Committed vs actual computed from PRNs and payments, shown at draft and at approval	Prevented	4



What still needs a person

Being honest about the remainder matters more than claiming the module solves everything. Three things stay human by design:

Deciding what a new name means

The system ranks candidates but cannot know a genuinely new vendor isn't a misspelling of an old one.

Confirming the paper exists

Only a person can confirm a signed PRN is in the folder. It becomes a tick, not a re-typing.

Explaining a cash difference

The cause is in the room, not the database. The module refuses to paper over it.



The measure of success

Not that the accounts desk has a new screen — but that the seven activities on page 21 reduce to three.

Collecting and chasing become a worklist that fills itself. Cleaning becomes a handful of queue decisions a week. Balancing disappears entirely.

What remains is the part that genuinely needs judgement: confirming the paper, and deciding the questions the system was right not to guess at.